

Unrolled List
–Search Element
–Space Complexity Analysis

Program:

```
// Function to search for an element
void searchElement(int val) {
    if (head == NULL) {
        cout << "The list is empty.\n";
        return;
    }

    struct Node* temp = head;
    int nodeCount = 1;

    while (temp != NULL) {
        for (int i = 0; i < temp->numElements; i++) {
            if (temp->elements[i] == val) {
                cout << "Element " << val << " found in Node " <<
nodeCount << " at index " << i << ".\n";
                return;
            }
        }
        temp = temp->next;
        nodeCount++;
    }

    cout << "Element " << val << " not found in the list.\n";
}

....

case 4:
    cout << "Enter integer value to search: ";
    cin >> value;
    searchElement(value);
    break;
```

Space Complexity Analysis

1. Input Data Space

The input space measures the memory required to store the data that the algorithm interacts with.

1. A. Parametric Input Space: $O(1)$

This refers exclusively to the memory needed for the arguments passed directly into the function signature.

- *The function signature is: `void searchElement(int val)`.*
- *It accepts exactly one parameter: a single integer `val`.*
- *Because a single integer requires a fixed, constant amount of memory (**typically 4 bytes**) regardless of how large the list grows, the memory footprint of the parameters never changes.*
- *Result: The parametric input space is strictly $O(1)$ (**Constant Space**).*

1. B. Full Data Holistic Space: $O(N)$ or $O(n)$

This measures the memory consumed by the actual data structure the algorithm is searching through.

- *The function searches a globally defined unrolled linked list accessed via the **head** pointer.*
- *To store **n** total elements, the structure requires $N \approx \left\lceil \frac{n}{4} \right\rceil$ nodes.*
- *Therefore, the total memory consumed by the data structure physically scales linearly with the number of elements.*

- **Result:** The holistic input space is $O(N)$ (which is mathematically equivalent to $O(n)$ i. e. $[O(N) \equiv O(n)]$).

2. Auxiliary Space Complexity: $O(1)$

Auxiliary Space measures the temporary, extra memory that the algorithm uses exclusively to perform its task, entirely separate from the input data.

*Let's trace every local variable created inside the **searchElement** block:*

- One traversal pointer: **`struct Node * temp = head;`**.
- One node tracker: **`int nodeCount = 1;`**.
- One loop iterator: **`int i = 0;`** (used inside the for loop).

Key Observations:

1. **No Dynamic Allocation:** The function only reads data. It never uses **`malloc()`** or **`new`** to create additional nodes or arrays.
2. **No Recursion:** The function relies on simple iterative loops (while and for), meaning the Call Stack does not grow.

The Call Stack depth remains at exactly 1.

Result: Because the extra memory required by the function is fixed and does not change based on the size of the list, i. e. independent of the input size **`n`**,

*The Auxiliary Space Complexity is strictly **$O(1)$** (Constant Space).*

Extra:

When, dynamically memory allocation to cause $O(N)$ auxiliary space, the amount of memory requested must scale up as the size of the list(N) scales up.

1. Looping N times to allocate N nodes.

```
// If a function did this, it would take  $O(N)$  auxiliary space
for (int i = 0; i < N; i++) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    // ... link the node ...
}
```

1. The Loop Execution

The for loop is governed by the condition $i < N$, meaning it will execute exactly N times.

2. The Dynamic Memory Allocation

*Inside the loop, the **malloc** function is called during every single iteration.*

***malloc** physically claims a brand – new*

block of memory from the computer's heap.

3. Total Memory Consumed

*If **sizeof(struct Node)** takes a fixed constant amount of memory (for example, 24 bytes), the total extra memory generated by this loop is:*

$$\text{Total Memory} = N \times 24 \text{ bytes.}$$

In Big – O notation, we drop the exact hardware constants (like 24 bytes) because we only care about how the memory requirements grow as N gets larger. Because the memory requested scales linearly with the variable N, the complexity is $O(N)$.

2. Allocating a massive block of memory in a single line:

```
struct Node* array_of_nodes = (struct Node*)malloc(N *  
sizeof(struct Node));
```

*Even though malloc is only called one time here,
the total size of the memory requested is multiplied by N.
If N gets larger, the memory block gets larger.*

$$\text{Space Complexity} = O(N)$$

For Academic Answer = Auxiliary Space = $O(1)$.

*"As we often cannot control the size of the input data, but we
can control how much extra memory of our algorithm
creates to process it."*

Total Space Complexity or Total memory footprint:

***= Input Space (Parametric Input Space + Full Data Holistic Input Space)
+ Auxiliary Space***

$$= (O(1) + O(N)) + O(1)$$

$$= O(N).$$

or,

$$(O(1) + O(n)) + O(1)$$

$$= O(n).$$

Key Notation Conventions

- **START**: Represents the global **head** pointer.
- **LINK[PTR]**: Represents the **next** pointer of a specific node block.
- **numElements[PTR] / elements[PTR][i]**: Represents accessing the attributes inside the specific node.

Algorithm: UnrolledSEARCHElem(val)

1. [CHECK IF LIST IS EMPTY]

- **IF START = NULL, Then:**
 - **Write: "The list is empty."**
 - **RETURN**
- **[End of If structure.]**

2. [INITIALIZE TRAVERSAL VARIABLES]

- **SET TEMP_PTR := START**
- **SET NODECOUNT := 1**

3. [OUTER LOOP: NODE TRAVERSAL]

- **REPEAT WHILE TEMP_PTR ≠ NULL:**

4. [INNER LOOP: ELEMENT SEARCH]

- **SET i := 0**
- **REPEAT WHILE i < numElements[TEMP_PTR]:**

5. [CHECK FOR MATCH]

- **IF elements[TEMP_PTR][i] = val, Then:**
 - **Write: "Element ", val, " found in Node ", NODECOUNT, " at index ", i, "."**
 - **RETURN (Exit the algorithm immediately upon success)**
- **[End of Match If structure.]**
- **SET i := i + 1 (Move to the next element in the current node)**
- **[End of Inner Search Loop]**

6. [MOVE TO NEXT NODE]

- **SET TEMP_PTR := LINK[TEMP_PTR] (Advance the pointer to the next node block)**
- **SET NODECOUNT := NODECOUNT + 1 (Increment the node tracker)**
- **[End of Outer Node Traversal Loop]**

7. [ELEMENT NOT FOUND]

*(If the outer loop finishes without
executing the inner return statement)*

- *Write: "Element ", val, " not found in the list."*

8. END OF ALGORITHM
